



UNIVERSIDAD POLITÉCNICA DE PACHUCA

INGENIERÍA EN SOFTWARE

ESTANCIA I

**PLAN DE MANTENIMIENTO DE SOFTWARE
BASADO EN SWEBOK V4.0 — ISO/IEC/IEEE 14764**

PROYECTO: YOLIZTLI

RESPONSABLE DE MANTENIMIENTO:

ISLAS LOZADA EMMANUEL

CATEDRÁTICO:

[NOMBRE DEL CATEDRÁTICO]

PACHUCA DE SOTO, HIDALGO 12 DE JUNIO DE 2026

Índice general

1. Introducción	3
1.1. Propósito del Documento	3
1.2. Alcance	3
1.3. Contexto del Proyecto	3
1.3.1. Descripción General del Sistema	3
1.3.2. Antecedentes de la Versión Anterior	4
1.3.3. Estado de las Pruebas al Inicio del Mantenimiento	4
2. Categorías de Mantenimiento	5
2.1. Mantenimiento Correctivo	5
2.2. Mantenimiento Preventivo	6
2.3. Mantenimiento Adaptativo	6
2.4. Mantenimiento Aditivo	7
2.5. Mantenimiento Perfectivo	7
3. Proceso de Mantenimiento	8
3.1. Ciclo de Vida de una Solicitud de Modificación (MR)	8
3.2. Convención de Commits	9
3.3. Gestión de la Configuración del Software (SCM)	9
4. Cronograma y Priorización de Actividades	10
4.1. Matriz de Prioridad de Solicitudes de Modificación	10
4.2. Descripción de Fases	10
4.2.1. Fase 1 — Estabilización (Corrección y Adaptación Base)	10
4.2.2. Fase 2 — Completitud Funcional (Nuevas Características y Robustez)	11
4.2.3. Fase 3 — Calidad Interna y Sostenibilidad	11
Referencias	12

Capítulo I

Introducción

I.1 PROPÓSITO DEL DOCUMENTO

El presente documento establece el Plan de Mantenimiento de Software para el sistema educativo **Yoliztli**, una plataforma web interactiva orientada a la enseñanza de lenguas originarias — náhuatl y otomí — para niños de educación primaria en el estado de Hidalgo.

El plan se rige bajo los lineamientos del estándar internacional **ISO/IEC/IEEE 14764** y la guía de conocimiento **SWEBOK V4.0a (Capítulo 7: Software Maintenance)**, los cuales definen las actividades, categorías y procesos requeridos para mantener un producto de software en operación continua y satisfactoria.

I.2 ALCANCE

Este documento aplica a la versión del sistema Yoliztli gestionada como *fork* del repositorio original. El alcance del mantenimiento comprende:

- La corrección de defectos reportados en el ciclo de pruebas previo.
- La transición de infraestructura tecnológica (motor de base de datos).
- La incorporación de nuevas funcionalidades requeridas por el SRS original no implementadas en la versión anterior.
- La mejora interna de la calidad y mantenibilidad del código.

I.3 CONTEXTO DEL PROYECTO

I.3.1 DESCRIPCIÓN GENERAL DEL SISTEMA

Yoliztli es una aplicación web construida sobre el stack tecnológico siguiente:

- **Backend:** Laravel 12 (PHP 8.2+), siguiendo el patrón de arquitectura MVC.

- **Frontend:** React 18 integrado mediante Inertia.js como adaptador SPA.
- **Base de Datos:** SQLite (desarrollo local) y PostgreSQL (producción).
- **Compilación:** Vite 5 con plugin de Laravel y React.
- **Estilización:** Tailwind CSS 3.

1.3.2 ANTECEDENTES DE LA VERSIÓN ANTERIOR

La versión original del sistema planteaba el uso de **Firebase** como plataforma backend (*Serverless*), con autenticación, base de datos en tiempo real y almacenamiento en la nube. El equipo de desarrollo del *fork* actual realizó una migración arquitectónica completa hacia una solución autohospedada con Laravel y SQLite, lo que otorga mayor independencia del entorno, control del flujo de datos y compatibilidad con infraestructura institucional propia.

1.3.3 ESTADO DE LAS PRUEBAS AL INICIO DEL MANTENIMIENTO

El reporte de calidad (*5.Pruebas Yoliztli.pdf*) ejecutado el 9 de diciembre de 2025 registra los siguientes resultados sobre 8 casos de prueba totales:

ID	Caso de Prueba	Resultado
CP001	Registro de usuario (nombre, correo, contraseña)	Aprobado
CP002	Inicio de sesión	Aprobado
CP003	Acceso a recursos interactivos	Aprobado
CP004	Selección de idioma de interfaz	Aprobado
CP005	Panel de reportes de docentes	Aprobado
CP006	Descarga de materiales educativos	Aprobado
CP007	Verificar filtro por categoría y edad	Fallido
CP008	Verificar el avance/progreso del usuario	Fallido

Tabla 1.1. Resultados del ciclo de pruebas de aceptación — Diciembre 2025

Los casos CP007 y CP008 representan la base de las actividades de **mantenimiento correctivo y aditivo** planificadas en este documento.

Capítulo 2

Categorías de Mantenimiento

De acuerdo con el estándar **ISO/IEC/IEEE 14764** y la sección 1.6 de SWEBOOK V4.0a, existen cinco categorías de mantenimiento que clasifican cualquier solicitud de modificación (Modification Request — MR). A continuación se presenta cada categoría junto con las actividades concretas identificadas para el proyecto Yoliztli.

2.1 MANTENIMIENTO CORRECTIVO

Modificación reactiva del software para corregir defectos descubiertos tras su entrega.

Las siguientes fallas fueron identificadas en el ciclo de pruebas de aceptación:

MR-Co1. Corrección del registro de avance del alumno (CPoo8)

Problema: El sistema no almacena correctamente el progreso de las actividades completadas por el alumno en la tabla progress de la base de datos. La prueba CPoo8 falló durante la validación.

Componentes afectados:

- app/Http/Controllers/ProgressController.php — método store().
- database/migrations/2025_12_05_055428_create_progress_table.php.
- Componentes React que invocan el endpoint POST /progress.

Acción: Depurar el flujo de petición HTTP entre el frontend y el controlador. Verificar que los campos user_id, item_id, type y score sean validados y persistidos correctamente.

MR-Co2. Resolución de conflicto de dependencias frontend (Inertia vs. React Router)

Problema: El archivo package.json declara simultáneamente react-router-dom e Inertia.js como estrategias de navegación, generando una contradicción arquitectónica que puede causar comportamientos indefinidos en la gestión de rutas.

Acción: Unificar la estrategia de navegación bajo **Inertia.js** (en concordancia con la implementación actual de los controladores PHP) y eliminar react-router-dom del árbol de dependencias.

2.2 MANTENIMIENTO PREVENTIVO

Modificaciones destinadas a detectar y corregir fallas latentes antes de que se manifiesten en el sistema en producción.

MR-Po1. Implementación de Form Requests para validación estricta

Objetivo: Crear clases de validación (Form Requests) para los endpoints `ProgressController` y `DocenteController`, mitigando riesgos de inyección de datos corruptos o malformados.

MR-Po2. Pruebas de regresión automatizadas

Objetivo: Implementar casos de prueba unitarios y de integración con **Pest** (instalado en `require-dev` del `composer.json`) para cubrir los flujos de inicio de sesión, registro de avance y consulta del panel de docentes. La suite de pruebas debe ejecutarse antes de cada fusión a la rama principal (`main`).

MR-Po3. Auditoría periódica de dependencias

Objetivo: Ejecutar `composer audit` y `npm audit` en cada ciclo de mantenimiento para identificar y mitigar vulnerabilidades de seguridad conocidas en las librerías del framework.

2.3 MANTENIMIENTO ADAPTATIVO

Modificaciones para mantener el software operativo ante cambios en el entorno tecnológico o de infraestructura.

MR-Ao1. Migración del motor de base de datos: SQLite a PostgreSQL

Objetivo: Cambiar la conexión de base de datos en el archivo `.env` y en `config/database.php` de SQLite a **PostgreSQL** para el despliegue en el entorno de producción institucional. Esta migración no requiere reescribir ninguna línea de lógica de negocio gracias a la abstracción de Eloquent ORM.

Pasos de implementación:

1. Instalar el servidor PostgreSQL en el entorno de producción.
2. Actualizar las variables de entorno `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME` y `DB_PASSWORD` en el archivo `.env`.
3. Ejecutar `php artisan migrate:fresh --seed` en el nuevo entorno.

MR-Ao2. Compatibilidad con actualizaciones de Laravel y Node.js

Objetivo: Monitorear los canales de publicación de *releases* de Laravel 12.x y Node.js LTS para adaptar la sintaxis de archivos de arranque (`bootstrap/app.php`) y configuración de Vite ante cambios de API mayores.

2.4 MANTENIMIENTO ADITIVO

Incorporación de nuevas funcionalidades no presentes en la versión recibida pero requeridas por el documento SRS original.

MR-AD01. Filtros por categoría y edad (CP007 — RF2)

Objetivo: Implementar la lógica de filtrado en `DashboardController` y los componentes React correspondientes para segmentar el contenido educativo (cuentos, videos, juegos) según el rango de edad del alumno y la categoría lingüística (Náhuatl / Otomí / Español).

Cambios requeridos en base de datos:

- Agregar el campo `age_range` (ej. 6-8, 9-12) en las tablas `stories` y `videos` mediante una nueva migración.
- Actualizar los seeders `StorySeeder` y `VideoSeeder` para incluir datos de clasificación por rango etario.

MR-AD02. Descarga de materiales en PDF (CP006 — RF4)

Objetivo: Implementar la generación dinámica de PDFs imprimibles para cuentos y actividades usando una librería compatible con Laravel (ej. `barryvdh/laravel-dompdf`). Crear una nueva ruta GET `/cuentos/{id}/pdf` con su controlador correspondiente.

2.5 MANTENIMIENTO PERFECTIVO

Mejoras orientadas a la eficiencia interna y la calidad del código sin alterar las funciones visibles del sistema.

MR-PF01. Optimización de consultas: Eager Loading en `DocenteController`

Objetivo: Refactorizar las consultas SQL en `DocenteController` para utilizar carga anticipada de relaciones (`with('progress')`) y eliminar el problema de rendimiento N+1 al listar alumnos y sus avances educativos.

MR-PF02. Limpieza de código JavaScript legado

Objetivo: Revisar y eliminar o migrar los archivos JavaScript residuales de la versión anterior (Vanilla JS) que permanecen en `resources/js/` (`document-ready.js`, `listbox-options.js`, `types.js`) y que no forman parte de la arquitectura React actual.

MR-PF03. Centralización de componentes de diseño en React

Objetivo: Crear una biblioteca de componentes reutilizables en `resources/js/Components/` (botones, tarjetas de contenido, indicadores de progreso) para garantizar uniformidad visual en todas las vistas de alumnos y docentes, siguiendo los lineamientos de diseño del documento *3.DiseñoInterfaces_Yoliztli.pdf*.

Capítulo 3

Proceso de Mantenimiento

La sección 2 del SWEBOK V4.0a describe el proceso de mantenimiento como un conjunto de actividades estructuradas que garantizan que cada modificación al software preserve su integridad. A continuación se define el proceso adoptado para el proyecto Yoliztli.

3.1 CICLO DE VIDA DE UNA SOLICITUD DE MODIFICACIÓN (MR)

Toda solicitud de modificación — ya sea una corrección de defecto (Problem Report — PR) o una mejora (Enhancement Request) — sigue el ciclo de seis etapas definido en ISO/IEC/IEEE 14764:

1. **Registro de la solicitud:** El problema o mejora se documenta en el gestor de issues del repositorio Git con una descripción clara, el componente afectado y el tipo de mantenimiento (Correctivo, Preventivo, Adaptativo, Aditivo o Perfectivo).
2. **Análisis de Impacto:** Antes de iniciar cualquier codificación, se evalúa cómo el cambio propuesto afecta a otros módulos del sistema, a la base de datos y a las pruebas existentes. Este análisis determina la estimación de esfuerzo.
3. **Diseño e Implementación:** El desarrollo se realiza en una rama Git dedicada siguiendo la convención de nombres:
 - `bugfix/<descripcion>` para mantenimiento correctivo.
 - `feat/<descripcion>` para mantenimiento aditivo.
 - `refactor/<descripcion>` para mantenimiento perfectivo.
 - `chore/<descripcion>` para mantenimiento adaptativo y preventivo.
4. **Revisión y Aceptación:** Se ejecuta la suite de pruebas automatizadas (`php artisan test`) y se verifica manualmente el caso de prueba afectado. El código se fusiona a `main` solo si todas las pruebas pasan.

5. **Migración y Despliegue:** Se aplican las nuevas migraciones de base de datos (`php artisan migrate`) en el entorno de destino y se recompila el frontend (`npm run build`).
6. **Cierre y Documentación:** El issue se cierra en el gestor de repositorio y se documenta el cambio en el historial de revisiones de este plan.

3.2 CONVENCIÓN DE COMMITS

Todos los commits del proyecto siguen la especificación de **Conventional Commits** para mantener un historial de cambios legible y estructurado:

Prefijo	Categoría SWEBOK	Ejemplo de uso
fix:	Correctivo	fix: corregir almacenamiento en tabla progress
feat:	Aditivo	feat: agregar filtros por edad en dashboard
refactor:	Perfectivo	refactor: optimizar consultas en DocenteController
chore:	Adaptativo / Preventivo	chore: migrar conexion a postgresql
docs:	Documentación	docs: actualizar plan de mantenimiento
test:	Preventivo	test: agregar pruebas para ProgressController

Tabla 3.1. Convención de mensajes de commit por categoría de mantenimiento

3.3 GESTIÓN DE LA CONFIGURACIÓN DEL SOFTWARE (SCM)

De acuerdo con la relación que SWEBOK establece entre mantenimiento y *Software Configuration Management* (SCM), el repositorio Git del proyecto aplica las siguientes prácticas:

- **Control de versiones centralizado:** Repositorio alojado en GitHub bajo la organización E33-a/Estancia-I.
- **Rama estable:** La rama main representa siempre el estado del sistema en producción. Ningún cambio se fusiona a esta rama sin haber pasado las pruebas de regresión.
- **Archivo .gitignore:** Excluye credenciales sensibles (`.env`), dependencias de terceros (`/vendor`, `/node_modules`) y archivos de base de datos locales (`*.sqlite`).
- **Etiquetado de versiones:** Cada entrega al catedrático se etiqueta con `git tag` siguiendo el esquema `v1.x.y` (versionado semántico).

Capítulo 4

Cronograma y Priorización de Actividades

4.1 MATRIZ DE PRIORIDAD DE SOLICITUDES DE MODIFICACIÓN

La siguiente tabla clasifica cada solicitud de modificación (MR) según su **prioridad** (Alta, Media, Baja), su **complejidad estimada** y la **fase** en que debe ejecutarse:

ID	Descripción	Tipo	Prioridad	Fase
MR-Co1	Corrección registro de avance (CPoo8)	Correctivo	Alta	1
MR-Co2	Resolver conflicto Inertia vs. React Router	Correctivo	Alta	1
MR-Ao1	Migración SQLite → PostgreSQL	Adaptativo	Alta	1
MR-Po1	Form Requests para validación de entradas	Preventivo	Media	2
MR-ADo1	Filtros por categoría y edad (CPoo7)	Aditivo	Alta	2
MR-ADo2	Descarga de materiales en PDF (RF4)	Aditivo	Media	2
MR-Po2	Pruebas de regresión con Pest	Preventivo	Media	2
MR-PFo1	Eager Loading en DocenteController	Perfectivo	Media	3
MR-PFo2	Limpieza de código JavaScript legado	Perfectivo	Baja	3
MR-PFo3	Centralización de componentes React	Perfectivo	Baja	3
MR-Po3	Auditoría periódica de dependencias	Preventivo	Baja	3
MR-Ao2	Seguimiento de actualizaciones de Laravel	Adaptativo	Baja	3

Tabla 4.1. Matriz de priorización de solicitudes de modificación

4.2 DESCRIPCIÓN DE FASES

4.2.1 FASE I — ESTABILIZACIÓN (CORRECCIÓN Y ADAPTACIÓN BASE)

Esta fase tiene como objetivo dejar el sistema en un estado funcional y ejecutable correctamente sobre la infraestructura de producción. Incluye las actividades de mayor urgencia identificadas a partir del ciclo

de pruebas:

- Resolución de la contradicción arquitectónica entre Inertia.js y React Router Dom en el frontend (MR-Co2).
- Depuración y corrección del flujo de almacenamiento de progreso (MR-Co1).
- Configuración y prueba de la conexión a PostgreSQL (MR-Ao1).

4.2.2 FASE 2 — COMPLETITUD FUNCIONAL (NUEVAS CARACTERÍSTICAS Y ROBUSTEZ)

Una vez estabilizado el núcleo del sistema, esta fase incorpora las funcionalidades que fallaron en las pruebas de aceptación y refuerza la seguridad del sistema:

- Implementación de los filtros de contenido por edad e idioma (MR-ADo1).
- Generación de PDFs de cuentos descargables (MR-ADo2).
- Implementación de validación estricta de peticiones (MR-Po1).
- Escritura de pruebas automatizadas de regresión (MR-Po2).

4.2.3 FASE 3 — CALIDAD INTERNA Y SOSTENIBILIDAD

Esta fase aborda la salud a largo plazo del código y la preparación del sistema para su evolución continua, conforme a las leyes de Lehman sobre evolución de software citadas en SWEBOK V4.0a:

- Optimización de rendimiento en consultas (MR-PFo1).
- Limpieza de código legado y reorganización de componentes (MR-PFo2, MR-PFo3).
- Establecimiento de proceso de auditoría de dependencias y seguimiento de versiones del framework (MR-Po3, MR-Ao2).

Referencias

- IEEE Computer Society. (2024). *SWEBOK Guide V4.0a, Chapter 7: Software Maintenance*. Institute of Electrical and Electronics Engineers.
- ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 14764:2022 — Software Engineering: Software Life Cycle Processes — Maintenance*. International Organization for Standardization.
- Lehman, M. M. (1980). *Programs, life cycles, and laws of software evolution*. Proceedings of the IEEE, 68(9), 1060–1076.
- Yoliztli Development Team. (2025). *Especificación de Requisitos de Software (SRS) — Revisión 1.0*. Universidad Politécnica de Pachuca.
- Galván, C. (2025). *Reporte de Pruebas Yoliztli — Corrida del 9 de diciembre de 2025*. Universidad Politécnica de Pachuca.